Exno: 6

Date: 8/8/24

Error Detection and Correction using Hamming Code.

Aim:

To write a program to implement error detection and correction using the Hamming Code.

Algorithm:

1. Start the program.
2. Read the no of 4-bit data blocks (n) forming the input data stream.
3. For each 4-bit data block D = d1 d2 d3 d4, perform Hamming encoding:
   a. Arrange 7 bit with parity bits at positions 1, 2 and 4 and bits at position 3, 5, 6 and 7.
   b. Compute the parity bits using even parity:
      P1 = d1 XOR d2 XOR d4 (covers bit position 1, 3, 5, 7)
      P2 = d1 XOR d3 XOR d4 (covers bit positions 2, 3, 6, 7)
      P4 = d2 XOR d3 XOR d4 (covers bit position 4, 5, 6, 7)
   c. Form the 7-bit Hamming code word P1 P2 d1 P4 d2 d3 d4
4. Simulate transmission by introducing a single-bit error at a chosen bit position in the code word.
5. At the receiver, decompute the check bits from the received code word:
   C1 = P1 XOR d1 XOR d2 XOR d4
   C2 = P2 XOR d1 XOR d3 XOR d4
   C4 = P4 XOR d2 XOR d3 XOR d4
6. Compute Error position = C4 C2 C1 4
7. If Error position = 0, no error has occured.
8. Extract the original 4 data bits from the corrected 7-bit code word.
9. Repeat step 3 to 8 for all n blocks in the data stream
10. Stop the program.

Program.

```
def encode(d):
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]

    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos} -> correcting...")
        h[error_pos-1] ^= 1
    else:
        print("No error detected.")
    return error_pos
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("===HAMMING CODE - ERROR DETECTION AND CORRECTION===\n")
    n = int(input("Enter number of 4-bit data blocks in the stream: "))
    for b in range(1, n+1):
        print(f"\n-- Block {b} --")
        d = list(map(int, input("Enter 4 data bits (space separated, e.g. 1 0 1 1): ").split()))

        h = encode(d)
        print(f"Original Data Bits  : {to_str(d)}")
        print(f"Encoded Hamming (7 bits): {to_str(h)}")
```

```
pos = int(input("Enter bit position to introduce error (1-7, 0=no error): "))
if 1<=pos<=7:
    h[pos-1] = 1
    print(f"Transmitted code(with error induced at pos {pos}): {''.join(str(b) for b in h)}")
else:
    print(f"Transmitted code(no error induced): {''.join(str(b) for b in h)}")
print(f"Received code      : {''.join(str(b) for b in h)}")
print("Checking for errors...")
detect_and_correct(h)
print(f"Corrected code     : {''.join(str(b) for b in h)}")
data = [h[2], h[4], h[5], h[6]]
print(f"Recovered Data Bits: {''.join(str(b) for b in data)}")
if __name__=="__main__":
    main()
```

Result:

The program to implement error detection and correction using the Hamming Code concept was successfully written in python, executed and verified using a sample input data stream

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===
...
Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 5
Transmitted Code (with error induced at pos 5): 0110111
Received Code : 0110111
Checking for errors...
Error detected at bit position: 5 -> Correcting...
Corrected Code : 0110011
Recovered Data Bits : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 0
Transmitted Code (no error induced): 0110011
Received Code : 0110011
Checking for errors...
No error detected.
Corrected Code : 0110011
Recovered Data Bits : 1011
```